

1. Course Overview

Duration

4–6 Hours

Objective

By the end of this session, learners should be able to:

- Understand databases and relational databases
 - Understand MySQL
 - Understand SQL and its categories
 - Create databases and tables
 - Understand data types
 - Insert, retrieve, update, and delete records
 - Use **SELECT** with filtering and sorting
 - Use operators
 - Use aggregate functions
 - Use **GROUP BY** and **HAVING**
 - Use different types of joins
 - Understand primary keys and foreign keys
 - Understand constraints
 - Use subqueries
 - Use views
 - Understand indexes
 - Understand transactions
 - Understand normalization basics
 - Create relationships between tables
 - Perform CRUD operations
 - Build an end-to-end Student Management Database
-

2. Recommended 4–6 Hour Session Plan

Session	Topic	Approx. Time
1	Database, DBMS, RDBMS, MySQL and SQL fundamentals	30 min
2	Database/Table creation and data types	40 min
3	INSERT, SELECT, UPDATE, DELETE	45 min
4	WHERE, operators, ORDER BY, LIMIT	35 min
5	Functions, GROUP BY and HAVING	35 min
6	Primary Key, Foreign Key and Constraints	30 min
7	Joins	45 min
8	Subqueries, Views and Indexes	30 min
9	Transactions and normalization	20 min
10	End-to-end mini project	45–60 min

The trainer can shorten or expand individual sections depending on learner experience.

PART 1 – DATABASE FUNDAMENTALS

3. What is Data?

Definition

Data is a collection of raw facts or information.

Examples:

101
John
25
Hyderabad
50000

Individually, these values may not provide much meaning.

When data is organized and processed, it becomes useful information.

Example

Raw data:

101, John, 25, Hyderabad

Organized information:

ID	Name	Age	City
101	John	25	Hyderabad

4. What is a Database?

Definition

A **database** is an organized collection of data that can be stored, accessed, managed, and retrieved efficiently.

Examples:

- Student database
- Banking database
- Employee database
- Hospital database
- E-commerce database

Example

A college database may contain:

Students
Courses
Faculty
Payments
Attendance
Examinations

5. What is DBMS?

Definition

DBMS stands for:

Database Management System

A DBMS is software used to create, store, retrieve, update, and manage data in databases.

Examples

- MySQL
- Oracle Database
- Microsoft SQL Server
- PostgreSQL
- SQLite

Main responsibilities of DBMS

Create database



Store data



Retrieve data



Update data



Delete data



Secure data



Manage users

6. What is RDBMS?

Definition

RDBMS stands for:

Relational Database Management System

An RDBMS stores data in **tables** and establishes relationships between tables.

Examples:

- MySQL
- PostgreSQL
- Oracle
- SQL Server

Example

Student table:

student_id	name	course_id
1	John	101
2	David	102

Course table:

course_id	course_name
101	Java
102	Python

The **course_id** connects the two tables.

7. What is MySQL?

Definition

MySQL is an open-source relational database management system that uses SQL to store and manage structured data.

MySQL is commonly used with:

- PHP
- Node.js
- Java
- Python
- .NET
- React + backend APIs

Example architecture

React / Angular / HTML



REST API



Node.js / Java / Python



MySQL



Database

8. What is SQL?

Definition

SQL stands for:

Structured Query Language

SQL is a language used to communicate with relational databases.

SQL allows us to:

- Create databases
- Create tables
- Insert records
- Retrieve records
- Update records
- Delete records
- Control access
- Manage transactions

Example:

```
SELECT * FROM students;
```

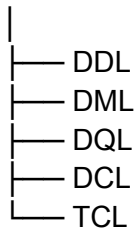
Meaning:

Retrieve all records from the students table.

9. SQL Command Categories

SQL commands are commonly divided into:

SQL



9.1 DDL

Definition

DDL = Data Definition Language

DDL is used to define or modify database structures.

Common commands:

CREATE
ALTER
DROP
TRUNCATE

Example:

```
CREATE TABLE students (  
    id INT,  
    name VARCHAR(100)  
);
```

9.2 DML

Definition

DML = Data Manipulation Language

DML is used to modify data stored inside tables.

Commands:

INSERT
UPDATE
DELETE

Example:

```
INSERT INTO students  
VALUES (1, 'John');
```

9.3 DQL

Definition

DQL = Data Query Language

Used to retrieve data.

Main command:

SELECT

Example:

```
SELECT * FROM students;
```

9.4 DCL

Definition

DCL = Data Control Language

Used to control database permissions.

Commands include:

GRANT
REVOKE

Example:

```
GRANT SELECT ON college.* TO 'student_user';
```

9.5 TCL

Definition

TCL = Transaction Control Language

Used to manage transactions.

Commands:

COMMIT
ROLLBACK
SAVEPOINT

Example:

```
START TRANSACTION;
```

```
UPDATE accounts  
SET balance = balance - 1000  
WHERE id = 1;
```

```
COMMIT;
```

PART 2 – MYSQL INSTALLATION AND TOOLS

10. MySQL Components

A typical MySQL setup includes:

MySQL Server



Stores databases and executes SQL

MySQL Client



Allows SQL commands

MySQL Workbench



Graphical interface for MySQL

MySQL Workbench

MySQL Workbench provides a GUI for:

- Creating databases
 - Creating tables
 - Running queries
 - Designing relationships
 - Viewing table structures
 - Managing users
-

11. Connecting to MySQL

Example command-line connection:

```
mysql -u root -p
```

Explanation:

mysql → MySQL client
-u → username
root → username
-p → ask for password

After entering the password:

```
mysql>
```

You can execute SQL commands.

PART 3 – DATABASE CREATION

12. What is a Database Schema?

Definition

A **schema** describes the structure of a database, including:

- Tables
- Columns
- Data types
- Relationships
- Constraints
- Views
- Indexes

In MySQL, the terms **database** and **schema** are often used interchangeably.

13. Create Database

Syntax:

```
CREATE DATABASE database_name;
```

Example:

```
CREATE DATABASE college;
```

14. Show Databases

```
SHOW DATABASES;
```

15. Select Database

```
USE college;
```

After this command, SQL operations are performed against the selected database.

16. Delete Database

```
DROP DATABASE college;
```

Warning

DROP DATABASE permanently removes the database and its objects.

PART 4 – TABLES

17. What is a Table?

Definition

A **table** is a structured collection of related data organized into rows and columns.

Example:

students

id	name	age	city
1	John	22	Hyderabad
2	David	23	Mumbai

18. What is a Row?

Definition

A **row** represents one complete record.

Example:

1 | John | 22 | Hyderabad

This represents one student.

19. What is a Column?

Definition

A **column** represents a specific attribute of a record.

Example:

id
name
age
city

20. Create Table

Syntax:

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype  
);
```

Example:

```
CREATE TABLE students (  
    id INT,  
    name VARCHAR(100),  
    age INT,  
    city VARCHAR(50)  
);
```

21. View Tables

SHOW TABLES;

22. View Table Structure

DESCRIBE students;

or:

DESC students;

PART 5 – MYSQL DATA TYPES

23. What is a Data Type?

Definition

A **data type** specifies what kind of value a column can store.

24. Numeric Data Types

INT

Stores whole numbers.

age INT

Example:

25
30
45

DECIMAL

Used for precise decimal values.

salary DECIMAL(10,2)

Example:

55000.50

DECIMAL(10,2) means:

- Maximum 10 digits
 - 2 digits after decimal
-

25. String Data Types

VARCHAR

Stores variable-length text.

name VARCHAR(100)

CHAR

Stores fixed-length strings.

gender CHAR(1)

TEXT

Used for larger text values.

description TEXT

26. Date and Time Types

DATE

Stores:

YYYY-MM-DD

Example:

2026-09-22

DATETIME

Stores date and time.

2026-09-22 10:30:00

TIMESTAMP

Stores date/time values and is commonly used for tracking creation/update timestamps.

27. Boolean

MySQL commonly represents Boolean values using:

BOOLEAN

which is effectively stored as a numeric type.

Example:

is_active BOOLEAN

Values:

TRUE

FALSE

PART 6 – CREATE A PRACTICAL TABLE

28. Student Table

Let's create a more realistic table.

```
CREATE TABLE students (  
    student_id INT,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    email VARCHAR(100),  
    age INT,  
    city VARCHAR(50),  
    course VARCHAR(50),  
    fees DECIMAL(10,2),  
    admission_date DATE  
);
```

PART 7 – CONSTRAINTS

29. What is a Constraint?

Definition

A **constraint** is a rule applied to table columns to control the data that can be stored.

Common constraints:

PRIMARY KEY
FOREIGN KEY
NOT NULL
UNIQUE
DEFAULT
CHECK

30. PRIMARY KEY

Definition

A **primary key** uniquely identifies each row in a table.

Example:

```
student_id INT PRIMARY KEY
```

Rules:

- Must be unique
- Cannot contain NULL
- One primary key constraint can identify each row

Example:

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(100)  
);
```

31. AUTO_INCREMENT

Definition

AUTO_INCREMENT automatically generates a new numeric value when a record is inserted.

Example:

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100)  
);
```

Insert:

```
INSERT INTO students (name)  
VALUES ('John');
```

MySQL automatically generates:

1

Next record:

2

32. NOT NULL

Definition

NOT NULL prevents a column from storing NULL values.

Example:

```
name VARCHAR(100) NOT NULL
```

33. UNIQUE

Definition

UNIQUE ensures that values in a column are not duplicated.

Example:

email VARCHAR(100) UNIQUE

Two students cannot have the same email.

34. DEFAULT

Definition

DEFAULT provides a value automatically when no value is supplied.

Example:

status VARCHAR(20) DEFAULT 'Active'

35. CHECK

Definition

CHECK defines a condition that values must satisfy.

Example:

age INT CHECK (age >= 18)

36. Complete Student Table

CREATE TABLE students (

```
student_id INT PRIMARY KEY AUTO_INCREMENT,  
first_name VARCHAR(50) NOT NULL,  
last_name VARCHAR(50),  
email VARCHAR(100) UNIQUE,  
age INT CHECK (age >= 18),  
city VARCHAR(50),  
course VARCHAR(50),  
fees DECIMAL(10,2),  
status VARCHAR(20) DEFAULT 'Active',  
admission_date DATE  
);
```

PART 8 – INSERT DATA

37. What is INSERT?

Definition

INSERT is used to add new records to a table.

Syntax:

```
INSERT INTO table_name  
(column1, column2)  
VALUES  
(value1, value2);
```

38. Insert One Record

INSERT INTO students


```
(first_name, last_name, email, age, city, course, fees, admission_date)
VALUES
(
    'John',
    'Smith',
    'john@gmail.com',
    22,
    'Hyderabad',
    'Node.js',
    45000,
    '2026-06-01'
);
```

39. Insert Multiple Records

```
INSERT INTO students
(first_name, last_name, email, age, city, course, fees, admission_date)
VALUES
('John','Smith','john@gmail.com',22,'Hyderabad','Node.js',45000,'2026-06-01'),
('David','Brown','david@gmail.com',24,'Mumbai','Java',50000,'2026-06-02'),
('Mary','Wilson','mary@gmail.com',21,'Chennai','Python',40000,'2026-06-03'),
('Robert','Taylor','robert@gmail.com',25,'Delhi','Node.js',45000,'2026-06-04'),
('Sara','Thomas','sara@gmail.com',23,'Hyderabad','Python',40000,'2026-06-05');
```

PART 9 – SELECT

40. What is SELECT?

Definition

SELECT is used to retrieve data from one or more tables.

41. Select All Columns

SELECT * FROM students;

* means all columns.

42. Select Specific Columns

SELECT first_name, email
FROM students;

43. Column Alias

Definition

An **alias** is a temporary name assigned to a column or table.

```
SELECT
    first_name AS Name,
    email AS EmailAddress
FROM students;
```

Output:

Name	EmailAddress
John	john@gmail.com

PART 10 – WHERE

44. What is WHERE?

Definition

WHERE filters records based on a condition.

Example:

```
SELECT *
FROM students
WHERE city = 'Hyderabad';
```

Only Hyderabad students are returned.

45. WHERE with Numbers

```
SELECT *  
FROM students  
WHERE age > 22;
```

46. Comparison Operators

Operator	Meaning
=	Equal
<>	Not equal
!=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Examples:

```
SELECT * FROM students WHERE age = 22;
```

```
SELECT * FROM students WHERE age > 22;
```

```
SELECT * FROM students WHERE age >= 23;
```

```
SELECT * FROM students WHERE city <> 'Hyderabad';
```

PART 11 – LOGICAL OPERATORS

47. AND

Definition

AND requires all conditions to be true.

```
SELECT *  
FROM students  
WHERE city = 'Hyderabad'  
AND age > 21;
```

48. OR

Definition

OR requires at least one condition to be true.

```
SELECT *  
FROM students  
WHERE city = 'Hyderabad'  
OR city = 'Mumbai';
```

49. NOT

Definition

NOT reverses a condition.

```
SELECT *  
FROM students  
WHERE NOT city = 'Hyderabad';
```

PART 12 – BETWEEN

50. BETWEEN

Definition

BETWEEN checks whether a value exists within a range.

```
SELECT *  
FROM students  
WHERE age BETWEEN 21 AND 24;
```

PART 13 – IN

51. IN

Definition

IN checks whether a value matches any value in a specified list.

Instead of:

```
WHERE city = 'Hyderabad'  
OR city = 'Mumbai'  
OR city = 'Delhi'
```

Use:

```
WHERE city IN ('Hyderabad', 'Mumbai', 'Delhi');
```

PART 14 – LIKE

52. LIKE

Definition

LIKE performs pattern matching.

%

Represents zero or more characters.

```
SELECT *  
FROM students  
WHERE first_name LIKE 'J%';
```

Matches:

John
James
Joseph

_

Represents exactly one character.

```
SELECT *  
FROM students  
WHERE first_name LIKE 'Jo_';
```

PART 15 – NULL

53. What is NULL?

Definition

NULL means the value is missing, unknown, or not provided.

Important:

NULL $\neq$ 0

NULL $\neq$ "

NULL $\neq$ FALSE

To check NULL:

```
SELECT *  
FROM students  
WHERE last_name IS NULL;
```

To check NOT NULL:

```
SELECT *  
FROM students  
WHERE last_name IS NOT NULL;
```

Do not use:

```
WHERE last_name = NULL
```

PART 16 – ORDER BY

54. What is ORDER BY?

Definition

ORDER BY sorts query results.

Ascending

```
SELECT *  
FROM students  
ORDER BY age ASC;
```

Descending

```
SELECT *  
FROM students  
ORDER BY age DESC;
```

ASC = ascending.

DESC = descending.

55. Multiple Column Sorting

```
SELECT *  
FROM students  
ORDER BY city ASC, age DESC;
```

First sorts by city.

Within each city, sorts by age descending.

PART 17 – LIMIT

56. LIMIT

Definition

LIMIT restricts the number of rows returned.

```
SELECT *  
FROM students  
LIMIT 3;
```

Returns the first three records.

57. LIMIT with OFFSET

```
SELECT *  
FROM students  
LIMIT 5 OFFSET 10;
```

Meaning:

Skip 10 records
Return next 5 records

This is commonly used for pagination.

PART 18 – UPDATE

58. What is UPDATE?

Definition

UPDATE modifies existing records.

Example:

```
UPDATE students  
SET city = 'Bangalore'  
WHERE student_id = 1;
```

Multiple columns

```
UPDATE students  
SET  
    city = 'Bangalore',  
    fees = 50000  
WHERE student_id = 1;
```

Important warning

Never accidentally execute:

```
UPDATE students  
SET city = 'Bangalore';
```

Without a **WHERE** clause, all records may be updated.

PART 19 – DELETE

59. What is DELETE?

Definition

DELETE removes records from a table.

```
DELETE FROM students  
WHERE student_id = 5;
```

Important warning

```
DELETE FROM students;
```

Deletes all rows.

PART 20 – DELETE vs TRUNCATE vs DROP

Command	Purpose
DELETE	Removes rows
TRUNCATE	Removes all rows
DROP	Removes table structure

DELETE

DELETE FROM students
WHERE student_id = 2;

TRUNCATE

TRUNCATE TABLE students;

DROP

DROP TABLE students;

Conceptually:

DELETE



Remove selected/all records

TRUNCATE



Remove all records but retain table

DROP



Remove table itself

PART 21 – SQL FUNCTIONS

60. What is a Function?

Definition

A **function** performs an operation and returns a result.

SQL functions are commonly divided into:

Aggregate Functions

+

String Functions

+

Numeric Functions

+

Date Functions

61. COUNT()

Definition

COUNT() returns the number of records.

```
SELECT COUNT(*)  
FROM students;
```

62. SUM()

Returns the total.

```
SELECT SUM(fees)  
FROM students;
```

63. AVG()

Returns the average.

```
SELECT AVG(fees)
FROM students;
```

64. MIN()

Returns the minimum value.

```
SELECT MIN(fees)
FROM students;
```

65. MAX()

Returns the maximum value.

```
SELECT MAX(fees)
FROM students;
```

66. Combined Example

```
SELECT
  COUNT(*) AS TotalStudents,
  SUM(fees) AS TotalFees,
  AVG(fees) AS AverageFees,
  MIN(fees) AS MinimumFees,
  MAX(fees) AS MaximumFees
FROM students;
```

PART 22 – GROUP BY

67. What is GROUP BY?

Definition

GROUP BY groups rows having the same value so aggregate calculations can be performed for each group.

Example:

```
SELECT
  city,
  COUNT(*) AS student_count
FROM students
GROUP BY city;
```

Output:

city	student_count
Hyderabad	2
Mumbai	1
Delhi	1

68. GROUP BY Course

```
SELECT
  course,
  COUNT(*) AS total_students
FROM students
GROUP BY course;
```

69. GROUP BY with SUM

```
SELECT
  course,
```



```
SUM(fees) AS total_fees  
FROM students  
GROUP BY course;
```

PART 23 – HAVING

70. What is HAVING?

Definition

HAVING filters groups after **GROUP BY**.

Example:

```
SELECT  
    course,  
    COUNT(*) AS total_students  
FROM students  
GROUP BY course  
HAVING COUNT(*) > 1;
```

71. WHERE vs HAVING

WHERE

Filters individual rows.

```
SELECT *  
FROM students  
WHERE age > 21;
```

HAVING

Filters groups.

```
SELECT course, COUNT(*)  
FROM students  
GROUP BY course  
HAVING COUNT(*) > 2;
```

Easy rule:

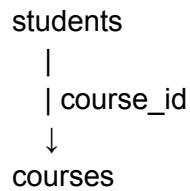
WHERE → before grouping
HAVING → after grouping

PART 24 – PRIMARY KEY AND FOREIGN KEY

72. What is a Relationship?

A relationship defines how tables are connected.

Example:



73. Foreign Key

Definition

A **foreign key** is a column that references a key in another table.

Create courses:

```
CREATE TABLE courses (  
  course_id INT PRIMARY KEY AUTO_INCREMENT,  
  course_name VARCHAR(100) NOT NULL,  
  duration INT,  
  fees DECIMAL(10,2)  
);
```

Modify students:

```
CREATE TABLE students (  
  student_id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100),  
  course_id INT,  
  FOREIGN KEY (course_id)  
    REFERENCES courses(course_id)  
);
```

74. Why Foreign Keys?

Foreign keys help maintain **referential integrity**.

Referential Integrity

It means relationships between related tables remain valid.

Example:

If:

`course_id = 101`

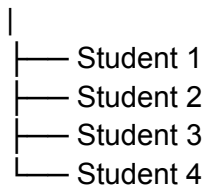
does not exist in the courses table, a student should not reference course 101 when foreign-key enforcement is active.

PART 25 – ONE-TO-MANY RELATIONSHIP

75. One-to-Many

One course can have many students.

Course



Database:

courses
course_id
course_name

students
student_id
name
course_id

PART 26 – JOINS

76. What is a JOIN?

Definition

A **JOIN** combines data from two or more tables based on a related column.

Suppose:

courses

course_id	course_name
1	Java
2	Python
3	Node.js

students

student_id	name	course_id
101	John	1
102	David	2
103	Sara	3

77. INNER JOIN

Definition

INNER JOIN returns only records that have matching values in both tables.

```
SELECT
  s.student_id,
  s.name,
  c.course_name
FROM students s
INNER JOIN courses c
  ON s.course_id = c.course_id;
```

Result:

student_id	name	course_name
101	John	Java
102	David	Python
103	Sara	Node.js

78. LEFT JOIN

Definition

LEFT JOIN returns all records from the left table and matching records from the right table.

```
SELECT
  s.name,
  c.course_name
FROM students s
LEFT JOIN courses c
  ON s.course_id = c.course_id;
```

If a student has no course, the course columns contain **NULL**.

79. RIGHT JOIN

Definition

RIGHT JOIN returns all records from the right table and matching records from the left table.

```
SELECT
  s.name,
  c.course_name
FROM students s
RIGHT JOIN courses c
  ON s.course_id = c.course_id;
```

80. CROSS JOIN

Definition

A **CROSS JOIN** produces combinations of every row from one table with every row from another table.

```
SELECT *
FROM students
CROSS JOIN courses;
```

If there are:

5 students
3 courses

Result:

$5 \times 3 = 15$ rows

81. SELF JOIN

Definition

A **self join** joins a table to itself.

Useful for hierarchical data.

Example:

employees

employee_id

name

manager_id

Query:

```
SELECT
  e.name AS Employee,
  m.name AS Manager
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.employee_id;
```

PART 27 – JOIN SUMMARY

Join	Meaning
INNER JOIN	Matching rows only
LEFT JOIN	All left + matching right
RIGHT JOIN	All right + matching left
CROSS JOIN	Every combination
SELF JOIN	Table joined with itself

PART 28 – SUBQUERIES

82. What is a Subquery?

Definition

A **subquery** is a query written inside another SQL query.

Example:

Find students whose fees are greater than average fees:

```
SELECT *  
FROM students  
WHERE fees > (  
    SELECT AVG(fees)  
    FROM students  
);
```

Inner query:

```
SELECT AVG(fees)  
FROM students;
```

Outer query:

```
SELECT *  
FROM students  
WHERE fees > ...
```

83. Subquery with IN

Find students enrolled in Python or Java courses using another table:

```
SELECT *  
FROM students  
WHERE course_id IN (  
    SELECT course_id  
    FROM courses  
    WHERE course_name IN ('Python', 'Java')  
);
```

PART 29 – EXISTS

84. What is EXISTS?

EXISTS checks whether a subquery returns at least one record.

Example:

```
SELECT *  
FROM courses c  
WHERE EXISTS (  
    SELECT 1  
    FROM students s  
    WHERE s.course_id = c.course_id  
);
```

This finds courses having at least one student.

PART 30 – CASE

85. What is CASE?

CASE provides conditional logic inside SQL.

Example:

```
SELECT
  name,
  age,
  CASE
    WHEN age < 21 THEN 'Young'
    WHEN age BETWEEN 21 AND 25 THEN 'Adult'
    ELSE 'Experienced'
  END AS AgeCategory
FROM students;
```

PART 31 – STRING FUNCTIONS

86. Common String Functions

CONCAT

Combines strings.

```
SELECT CONCAT(first_name, ' ', last_name)
FROM students;
```

UPPER

```
SELECT UPPER(first_name)
FROM students;
```

LOWER

```
SELECT LOWER(first_name)
FROM students;
```

LENGTH

```
SELECT LENGTH(first_name)
FROM students;
```

TRIM

Removes unnecessary spaces.

```
SELECT TRIM(name)
FROM students;
```

PART 32 – DATE FUNCTIONS

87. Common Date Functions

Current date

```
SELECT CURDATE();
```

Current date and time

```
SELECT NOW();
```

Extract year

```
SELECT YEAR(admission_date)
FROM students;
```

Extract month

```
SELECT MONTH(admission_date)
FROM students;
```

Date difference

```
SELECT DATEDIFF(CURDATE(), admission_date)
FROM students;
```

PART 33 – INDEXES

88. What is an Index?

Definition

An **index** is a database structure that helps MySQL find rows more efficiently.

Think of a book.

Without an index:

Read every page



Find topic

With an index:

Check index



Find page



Read data

89. Create Index

```
CREATE INDEX idx_student_city  
ON students(city);
```

Now queries filtering by city can potentially benefit from this index.

90. Unique Index

```
CREATE UNIQUE INDEX idx_student_email  
ON students(email);
```

This also prevents duplicate email values.

91. Index Disadvantage

Indexes improve read performance but require:

- Storage
- Maintenance
- Additional work during INSERT
- Additional work during UPDATE
- Additional work during DELETE

Therefore, do not blindly index every column.

PART 34 – VIEWS

92. What is a View?

Definition

A **view** is a virtual table based on a SQL query.

Example:

```
CREATE VIEW student_course_view AS
SELECT
    s.student_id,
    s.name,
    c.course_name
FROM students s
JOIN courses c
    ON s.course_id = c.course_id;
```

Use:

```
SELECT *
FROM student_course_view;
```

93. Why Use Views?

Views can help with:

- Simplifying complex queries
 - Reusing queries
 - Restricting exposed columns
 - Creating reporting layers
-

PART 35 – TRANSACTIONS

94. What is a Transaction?

Definition

A **transaction** is a group of database operations treated as one logical unit.

Example:

Bank transfer:

Account A

↓

Deduct ₹1,000

Account B

↓

Add ₹1,000

Both operations should succeed together.

95. START TRANSACTION

START TRANSACTION;

Perform operations:

```
UPDATE accounts  
SET balance = balance - 1000  
WHERE account_id = 1;
```

```
UPDATE accounts  
SET balance = balance + 1000  
WHERE account_id = 2;
```

Commit:

COMMIT;

96. ROLLBACK

Definition

ROLLBACK reverses changes made during the current transaction that have not been committed.

START TRANSACTION;

```
UPDATE accounts  
SET balance = balance - 1000  
WHERE account_id = 1;
```

ROLLBACK;

The change is rolled back.

97. SAVEPOINT

Definition

A **SAVEPOINT** creates a point within a transaction to which you can roll back.

```
START TRANSACTION;
```

```
UPDATE students  
SET fees = fees + 1000  
WHERE student_id = 1;
```

```
SAVEPOINT point1;
```

```
UPDATE students  
SET fees = fees + 2000  
WHERE student_id = 2;
```

```
ROLLBACK TO point1;
```

```
COMMIT;
```

PART 36 – ACID PROPERTIES

98. What is ACID?

ACID describes important properties of database transactions.

A → Atomicity
C → Consistency
I → Isolation
D → Durability

Atomicity

A transaction is treated as one unit.

All operations succeed
OR
All operations fail

Consistency

The database moves from one valid state to another valid state.

Isolation

Transactions should not improperly interfere with each other.

Durability

Once committed, data should persist even after system failures.

PART 37 – NORMALIZATION

99. What is Normalization?

Definition

Normalization is the process of organizing database data to reduce duplication and improve data integrity.

Bad design:

student_id
student_name
course1
course2
course3

Better design:

students
courses
student_courses

100. First Normal Form – 1NF

A table should contain atomic values.

Bad:

student	phones
John	9876, 8765

Better:

student	phone
John	9876
John	8765

101. Second Normal Form – 2NF

A table should:

- Be in 1NF
- Have no partial dependency on part of a composite key

This becomes particularly important when a table has a composite primary key.

102. Third Normal Form – 3NF

A table should:

- Be in 2NF
- Avoid transitive dependencies

Example:

Bad:

student_id
student_name
course_id
course_name

Better:

students
student_id
student_name
course_id

courses
course_id
course_name

PART 38 – STORED PROCEDURES

103. What is a Stored Procedure?

Definition

A **stored procedure** is a group of SQL statements stored inside the database and executed when called.

Example:

```
DELIMITER //
```

```
CREATE PROCEDURE GetStudents()  
BEGIN  
    SELECT *  
    FROM students;  
END //
```

```
DELIMITER ;
```

Execute:

```
CALL GetStudents();
```

104. Stored Procedure with Parameter

```
DELIMITER //
```

```
CREATE PROCEDURE GetStudentById(IN p_id INT)  
BEGIN  
    SELECT *  
    FROM students  
    WHERE student_id = p_id;
```

END //

DELIMITER ;

Call:

CALL GetStudentById(1);

PART 39 – SQL QUERY EXECUTION ORDER

105. Important SQL Concept

Although we write:

SELECT
FROM
WHERE
GROUP BY
HAVING
ORDER BY
LIMIT

the logical processing order is approximately:

FROM
↓
WHERE
↓
GROUP BY
↓
HAVING
↓
SELECT
↓
ORDER BY
↓
LIMIT

Example:

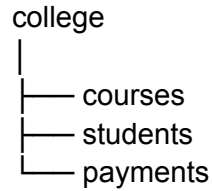
```
SELECT
    course,
    COUNT(*) AS total
FROM students
WHERE age >= 21
GROUP BY course
HAVING COUNT(*) > 1
ORDER BY total DESC
LIMIT 5;
```

Understanding this order is very useful when debugging SQL queries.

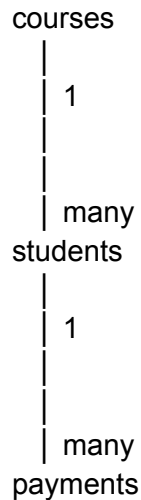
PART 40 – END-TO-END MINI PROJECT

106. Student Management Database

We will create:



Relationships:



107. Create Database

```
CREATE DATABASE college;
```

```
USE college;
```

108. Create Courses Table

```
CREATE TABLE courses (  
  course_id INT PRIMARY KEY AUTO_INCREMENT,  
  course_name VARCHAR(100) NOT NULL,  
  duration_months INT,  
  fees DECIMAL(10,2)
```

);

109. Insert Courses

```
INSERT INTO courses
(course_name, duration_months, fees)
VALUES
('Java', 6, 50000),
('Python', 5, 45000),
('Node.js', 4, 40000),
('React', 3, 35000);
```

110. Create Students

```
CREATE TABLE students (
  student_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE,
  age INT,
  city VARCHAR(50),
  course_id INT,
  admission_date DATE,
  status VARCHAR(20) DEFAULT 'Active',

  FOREIGN KEY (course_id)
    REFERENCES courses(course_id)
);
```

111. Insert Students

```
INSERT INTO students
(name, email, age, city, course_id, admission_date)
VALUES
('John Smith', 'john@gmail.com', 22, 'Hyderabad', 1, '2026-06-01'),
```

```
('David Brown','david@gmail.com',24,'Mumbai',2,'2026-06-05'),  
('Mary Wilson','mary@gmail.com',21,'Hyderabad',3,'2026-06-10'),  
('Robert Taylor','robert@gmail.com',25,'Delhi',1,'2026-06-15'),  
('Sara Thomas','sara@gmail.com',23,'Chennai',4,'2026-06-20');
```

112. Create Payments Table

```
CREATE TABLE payments (  
    payment_id INT PRIMARY KEY AUTO_INCREMENT,  
    student_id INT,  
    amount DECIMAL(10,2),  
    payment_date DATE,  
    payment_status VARCHAR(20),  
  
    FOREIGN KEY (student_id)  
        REFERENCES students(student_id)  
);
```

113. Insert Payments

```
INSERT INTO payments  
(student_id, amount, payment_date, payment_status)  
VALUES  
(1,25000,'2026-06-05','Paid'),  
(1,15000,'2026-07-05','Paid'),  
(2,45000,'2026-06-10','Paid'),  
(3,20000,'2026-06-15','Paid'),  
(4,25000,'2026-06-20','Paid'),  
(5,35000,'2026-06-25','Paid');
```

114. Retrieve All Students

```
SELECT *  
FROM students;
```

115. Display Student and Course

```
SELECT
    s.student_id,
    s.name,
    s.email,
    c.course_name,
    c.fees
FROM students s
JOIN courses c
    ON s.course_id = c.course_id;
```

116. Find Hyderabad Students

```
SELECT *
FROM students
WHERE city = 'Hyderabad';
```

117. Students Above Age 22

```
SELECT *
FROM students
WHERE age > 22;
```

118. Sort Students by Age

```
SELECT *
FROM students
ORDER BY age DESC;
```

119. Count Students

```
SELECT COUNT(*) AS total_students
FROM students;
```

120. Students Per Course

```
SELECT
    c.course_name,
    COUNT(s.student_id) AS total_students
FROM courses c
LEFT JOIN students s
    ON c.course_id = s.course_id
GROUP BY c.course_id, c.course_name;
```

121. Total Payments

```
SELECT
    SUM(amount) AS total_payment
FROM payments;
```

122. Payment Per Student

```
SELECT
    s.name,
    SUM(p.amount) AS total_paid
FROM students s
JOIN payments p
    ON s.student_id = p.student_id
GROUP BY s.student_id, s.name;
```

123. Student Payment Report

```
SELECT
    s.student_id,
    s.name,
    c.course_name,
    c.fees AS course_fees,
```

```
COALESCE(SUM(p.amount), 0) AS amount_paid,  
c.fees - COALESCE(SUM(p.amount), 0) AS balance  
FROM students s  
JOIN courses c  
  ON s.course_id = c.course_id  
LEFT JOIN payments p  
  ON s.student_id = p.student_id  
GROUP BY  
  s.student_id,  
  s.name,  
  c.course_name,  
  c.fees;
```

What is COALESCE?

COALESCE() returns the first non-NULL value.

Example:

```
COALESCE(NULL, 0)
```

returns:

0

Therefore, students with no payments can be treated as having a payment amount of zero.

PART 41 – PRACTICAL SQL CRUD

124. What is CRUD?

CRUD represents four fundamental database operations:

C → Create
R → Read
U → Update
D → Delete

Create

```
INSERT INTO students (...)  
VALUES (...);
```

Read

```
SELECT *  
FROM students;
```

Update

```
UPDATE students  
SET city = 'Pune'  
WHERE student_id = 1;
```

Delete

```
DELETE FROM students  
WHERE student_id = 1;
```

PART 42 – SQL WITH APPLICATIONS

125. Typical Backend Architecture

For a Node.js application:

React Frontend
↓
HTTP Request
↓
Express API
↓
Controller
↓
Service
↓
MySQL Driver
↓
MySQL Database

Example:

GET /api/students
↓
Controller
↓
Service
↓
SELECT * FROM students
↓
MySQL
↓
JSON Response

126. Parameterized Queries

When applications receive user input, avoid constructing SQL like:

```
const sql =  
  "SELECT * FROM students WHERE email = " + email + "";
```

This can create SQL injection vulnerabilities.

Instead use parameterized queries.

Example with Node.js/MySQL:

```
const sql = `
  SELECT *
  FROM students
  WHERE email = ?
`;
```

```
const [rows] = await connection.execute(sql, [email]);
```

The `?` is a parameter placeholder.

PART 43 – SQL INJECTION

127. What is SQL Injection?

Definition

SQL Injection is a security vulnerability where malicious SQL input changes the intended database query.

Unsafe:

```
const query =  
  "SELECT * FROM users WHERE username = '" +  
  username +  
  "'";
```

Secure approach:

```
const query =  
  "SELECT * FROM users WHERE username = ?";
```

Use parameterized queries/prepared statements.

PART 44 – COMMON SQL ERRORS

128. Forgetting WHERE

Dangerous:

```
UPDATE students  
SET city = 'Delhi';
```

This may update every student.

Correct:

```
UPDATE students  
SET city = 'Delhi'  
WHERE student_id = 5;
```

129. Incorrect NULL Comparison

Wrong:

```
WHERE city = NULL;
```

Correct:

```
WHERE city IS NULL;
```

130. Incorrect GROUP BY

Example:

```
SELECT city, name, COUNT(*)  
FROM students
```

GROUP BY city;

Depending on SQL mode and selected columns, **name** may not be valid because it is neither grouped nor aggregated.

Better:

```
SELECT city, COUNT(*)  
FROM students  
GROUP BY city;
```

131. Duplicate Records

Use:

```
SELECT DISTINCT city  
FROM students;
```

DISTINCT

DISTINCT removes duplicate result values.

PART 45 – IMPORTANT SQL CONCEPTS

132. Primary Key vs Foreign Key

Primary Key	Foreign Key
Uniquely identifies row	References another table
Cannot be NULL	Can be NULL unless restricted
Usually one primary key constraint	Multiple foreign keys possible
Maintains entity identity	Maintains relationships

133. DELETE vs TRUNCATE vs DROP

Feature	DELETE	TRUNCATE	DROP
Removes rows	Yes	Yes	Yes
Removes table	No	No	Yes
WHERE supported	Yes	No	No
Table structure retained	Yes	Yes	No
Typical use	Selective deletion	Clear table	Remove object

134. WHERE vs HAVING

WHERE	HAVING
Filters rows	Filters groups

Used before grouping

Used after grouping

Can be used without GROUP BY

Usually associated with GROUP BY

Example: age > 20

Example: COUNT(*) > 5

135. CHAR vs VARCHAR

CHAR

Fixed-length

Useful for fixed-size values

Can use fixed storage characteristics

Examples:

gender CHAR(1)

name VARCHAR(100)

VARCHAR

Variable-length

Useful for normal text

Stores variable-length strings

PART 46 – IMPORTANT SQL COMMAND CHEAT SHEET

Database

CREATE DATABASE college;

SHOW DATABASES;

USE college;

DROP DATABASE college;

Tables

CREATE TABLE students (...);

SHOW TABLES;

DESC students;

ALTER TABLE students ADD phone VARCHAR(20);

DROP TABLE students;

TRUNCATE TABLE students;

Data

INSERT INTO students (...) VALUES (...);

SELECT * FROM students;

UPDATE students
SET city = 'Delhi'
WHERE student_id = 1;

DELETE FROM students
WHERE student_id = 1;

Filtering

WHERE
AND
OR
NOT
IN
BETWEEN
LIKE
IS NULL
IS NOT NULL

Sorting

ORDER BY age ASC;

ORDER BY age DESC;

Grouping

GROUP BY city;

HAVING COUNT(*) > 2;

Aggregate Functions

COUNT()

SUM()

AVG()

MIN()

MAX()

Joins

INNER JOIN

LEFT JOIN

RIGHT JOIN

CROSS JOIN

SELF JOIN

Transactions

START TRANSACTION;

COMMIT;

ROLLBACK;

SAVEPOINT;

PART 47 – PRACTICAL EXERCISES

Exercise 1 – Database

Create:

company

Create a table:

employees

Columns:

employee_id

name

email

age

department

salary

joining_date

Exercise 2 – Insert

Insert at least 10 employees.

Exercise 3 – Select

Display:

All employees

Only names

Names and salaries

Exercise 4 – Filtering

Find:

Employees older than 30
Employees earning more than 50000
Employees from IT department
Employees from IT or HR

Exercise 5 – Sorting

Find employees:

Sorted by salary ascending
Sorted by salary descending
Sorted by name

Exercise 6 – Aggregate Functions

Find:

Total employees
Total salary
Average salary
Highest salary
Lowest salary

Exercise 7 – Grouping

Display:

Number of employees per department
Average salary per department
Highest salary per department

Exercise 8 – HAVING

Find departments having more than 3 employees.

Exercise 9 – Joins

Create:

departments
employees

Create a relationship and display:

Employee Name
Department Name
Salary

PART 48 – INTERVIEW QUESTIONS

Beginner

1. What is SQL?

SQL is a language used to communicate with relational databases.

2. What is MySQL?

MySQL is a relational database management system that uses SQL.

3. What is a database?

An organized collection of data.

4. What is a table?

A structured collection of related records arranged in rows and columns.

5. What is a primary key?

A column or set of columns that uniquely identifies a record.

6. What is a foreign key?

A column that references a key in another table.

7. What is NULL?

A missing or unknown value.

8. What is CRUD?

Create
Read
Update
Delete

PART 49 – INTERMEDIATE INTERVIEW QUESTIONS

9. Difference between DELETE and TRUNCATE?

DELETE can remove selected records using **WHERE**, whereas **TRUNCATE** removes all records from a table.

10. Difference between WHERE and HAVING?

WHERE filters rows, while **HAVING** filters groups.

11. What is JOIN?

JOIN combines rows from multiple tables based on related columns.

12. What is normalization?

Normalization organizes data to reduce redundancy and improve data integrity.

13. What is an index?

An index is a data structure used to improve query performance.

14. What is a view?

A view is a virtual table based on a query.

15. What is a transaction?

A transaction is a group of operations treated as a single logical unit.

PART 50 – ADVANCED PRACTICE QUERIES

Query 1

Find the second-highest salary:

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

Query 2

Find employees earning above average salary:

```
SELECT *  
FROM employees  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employees  
);
```

Query 3

Find departments having average salary greater than 60000:

```
SELECT  
    department,  
    AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department  
HAVING AVG(salary) > 60000;
```

Query 4

Find employees whose names start with J:

```
SELECT *  
FROM employees  
WHERE name LIKE 'J%';
```

Query 5

Find duplicate emails:

```
SELECT  
    email,  
    COUNT(*) AS total  
FROM employees  
GROUP BY email
```

HAVING COUNT(*) > 1;

PART 51 – END-TO-END SQL LEARNING FLOW

A learner should understand SQL in this order:

1. Database
- ↓
2. DBMS
- ↓
3. RDBMS
- ↓
4. MySQL
- ↓
5. SQL
- ↓
6. Database Creation
- ↓
7. Tables
- ↓
8. Data Types

↓
9. Constraints
↓
10. INSERT
↓
11. SELECT
↓
12. WHERE
↓
13. Operators
↓
14. ORDER BY
↓
15. LIMIT
↓
16. UPDATE
↓
17. DELETE
↓
18. Functions
↓
19. GROUP BY
↓
20. HAVING
↓
21. Primary Key
↓
22. Foreign Key
↓
23. Relationships
↓
24. JOIN
↓
25. Subqueries
↓
26. Views
↓
27. Indexes
↓
28. Transactions
↓
29. Normalization
↓
30. Real Project

PART 52 – FINAL END-TO-END EXERCISE

Create a database:

training_center

Create the following tables:

students

courses

payments

Students

student_id

name

email

phone

city

course_id

admission_date

status

Courses

course_id
course_name
duration
fees

Payments

payment_id
student_id
amount
payment_date
payment_status

Implement:

Task 1

Create all three tables.

Task 2

Add primary keys.

Task 3

Add foreign keys.

Task 4

Insert at least:

5 courses
10 students
15 payments

Task 5

Display all students.

Task 6

Display students with their course names.

Task 7

Find students from Hyderabad.

Task 8

Find students whose age is greater than 21.

Task 9

Find the number of students per course.

Task 10

Find the total amount collected.

Task 11

Find the amount paid by each student.

Task 12

Find students who have not made any payment.

Task 13

Find the highest course fee.

Task 14

Find the average course fee.

Task 15

Find courses having more than 2 students.

Task 16

Create a view for the student-course report.

Task 17

Create an index on student email.

Task 18

Create a transaction for a payment operation.

